

Chapter 24: Scripting with Kotlin

By Ellen Shapiro

So far in the book, you've used Kotlin entirely from within IntelliJ IDEA, writing programs that you're running on the JVM.

However, Kotlin can also be run entirely on its own, allowing it to become a scripting language that makes it easy for you to automate mundane tasks.

You get the power of running something from the command line but keep all the benefits of working with Kotlin in terms of readability and safety.

IMPORTANT: The remainder of this chapter assumes that you are running either macOS, Linux or some other Kotlin-supported Unix operating system (ex., FreeBSD or Solaris). If you're running Windows, you'll want to use either [Cygwin](https://www.cygwin.com/) (<https://www.cygwin.com/>) or the Windows 10 [Subsystem for Linux](https://docs.microsoft.com/en-us/windows/wsl/install-win10) (<https://docs.microsoft.com/en-us/windows/wsl/install-win10>) to be able to use the same commands shown here. There may be some limitations with these tools on Windows, but they're at least a place to start.

What is scripting?

Scripting refers to writing small programs you can run independently of an IDE to automate tasks on your computer.

A **script** is the small program that you write and run. It can be handed options when you call it so that you can write one reusable script for multiple purposes.

You'll often hear people talk about **shell scripting**, which is using `.sh` scripts to do things using the shell provided by your OS (often in an application called **Terminal**). Common shells are **bash**, **zsh**, and **fish**.

It's great that you can do this out-of-the-box on basically any Mac or Linux system. However, there are a number of issues with shell scripting that have led developers to pursue alternatives:

- Shell scripting is **not type-safe**. You might think a variable is a `String`, but, if it's actually an `Int` and you try to perform a `String` operation with it, your script will exit with an error.
- Shell scripting is **not compiled**. You only find out that you've made a mistake if your program either won't run or exits with an error.
- Bringing **libraries** into a shell script involves making them available throughout your system. This may not be behavior you want for many reasons, including security.
- Shell scripts can be very **difficult to read**. Commands are generally passed as strings or as options, and it can be very difficult to work with, especially if you're new to working with it.

Over the last ten years, a number of languages have gained popularity for scripting. **Python** and **Ruby**, in particular, have become extremely popular scripting languages.

However, while scripts written in either of those languages are vastly more readable than shell scripts, and they make bringing in libraries far safer, neither Python nor Ruby are type-safe in the way that Kotlin is. Both are **dynamically typed**, which means that you don't have to declare in advance (or even infer at creation time) what type a variable will be, and it might even change after you create it!

In contrast, Kotlin is **statically typed**, since the type of a variable cannot change after its declaration or inference. For example, when you write the following:

```
val three = 3
```

The variable `three` is *inferred* to have a type of `Int`. If you were to write:

```
val three: String = 3
```

You'd get an error, since you've explicitly declared the type of `three` to be a `String`, and the value `3` is not a `String`, it's an `Int`. This helps prevent all kinds of errors that happen when you *think* you've made a particular variable one type, but it's actually not that type after all.

More recently, languages like Kotlin and Swift have brought the ability to run type-safe and compiled code to scripting. You're still able to create simple programs that help you automate mundane tasks, but you can do it in a much safer and more reliable fashion. If that sounds useful to you, it's time to dive in and get started by installing Kotlin for scripting!

Installing Kotlin for scripting

Up to this point, your computer has been accessing Kotlin through your IDE, IntelliJ IDEA. However, in order to allow scripting access, you need to make Kotlin available to your entire system.

To do this, you're going to use a tool that allows management of SDKs for macOS and Linux called SDKMan! (<https://sdkman.io>) to install Kotlin for the command line.

Note: Alternatively, there's a tutorial on the Kotlin website that helps walk you through other ways to install for macOS (such as Homebrew and MacPorts) and Ubuntu systems (Snap), and that also gives you instructions for a manual install. You can find the tutorial at <https://kotlinlang.org/docs/tutorials/command-line.html> and at <https://brew.sh>.

You **will** need an Internet connection for the rest of these installation steps.

Installing SDKMan!

Note: If you’ve already got SDKMan! installed, skip to the “Installing Kotlin” section below.

If you’re on macOS or Ubuntu Linux, open up the **Terminal** program. When it finishes loading, type in:

```
curl -s https://get.sdkman.io | bash
```

Press **Return** on Mac or **Enter** on PC (we’ll call this “Press Enter” for the rest of the chapter). This will go to <https://get.sdkman.io> and install the latest version, which works with bash — the main command line shell on many Unix systems, including macOS and Linux.

Note: If you’re on non-Ubuntu flavors of Linux, any time this chapter references Terminal, it’s referring to a bash shell. If you’re running a flavor of Linux other than Ubuntu, you’re most likely familiar with how your system allows input into a bash shell. Please use that instead of the Terminal application whenever Terminal is referenced.

IMPORTANT: There may be some instructions that print out when the curl command finishes — these differ a bit from system to system. If there are any instructions printed out, please follow them before considering the installation complete and proceeding.

Once your installation of SDKMan! is complete, it’s time to actually install Kotlin.

Installing Kotlin

Open a new Terminal — either a new window or a new tab if your shell program allows it — and install Kotlin using SDKMan! by typing the following command and pressing Enter:

```
sdk install kotlin
```

This will use SDKMan! to fetch the most recent binary of Kotlin, install it on your machine and make it available within your shell \$PATH, which means that you will be able to execute Kotlin from any directory in your system.

Once the installation is complete, enter the following at the command line to validate that Kotlin is installed and available to your entire system:

```
which kotlin
```

This should print out a path similar to this one, replacing [username] with your actual username:

```
/Users/[username]/.sdkman/candidates/kotlin/current/bin/kotlin
```

Or this one:

```
/usr/local/bin/kotlin
```

Seeing this confirms that Kotlin was installed using SDKMan! and that it should be accessible to you whenever you're using Terminal.

To make sure everything is working correctly, type the following at the command line:

```
kotlinc-jvm
```

This will launch a **Read-Evaluate-Print-Loop**, or REPL.

Using the REPL

A REPL is essentially a tiny Kotlin program in which you can type things and have them execute immediately. When it launches, you'll see something like this:

```
ellen@MacBook-Pro Book % kotlinc-jvm
Welcome to Kotlin version 1.4.21 (JRE 1.8.0_91-b14)
Type :help for help, :quit for quit
>>> █
```

Next to the >>> prompt where you see a blinking cursor, type in the following, then press Enter:

```
println("Hello, world!")
```

The way that a REPL works is fairly straightforward:

- It **reads** whatever you've entered at the prompt.
- It then **evaluates** whatever you've typed as if it were the contents of a Kotlin function.
- It then **prints** the results of the evaluation.
- Finally, it **loops** back around to the beginning, so it can start reading again when you enter something new.

In this case, printing the results of the evaluated `println` statement results in (surprise!) that statement being printed out:

```
ellen@MacBook-Pro Book % kotlinc-jvm
Welcome to Kotlin version 1.4.21 (JRE 1.8.0_91-b14)
Type :help for help, :quit for quit
>>> println("Hello, world!")

Hello, world!
>>>
>>> █
```

The REPL is also capable of more complex operations, and it has the full ability of the compiler to use type inference, though you'll notice very quickly that it lacks autocomplete.

It also has the ability to hold on to defined variables and constants that are used throughout a single session — from when you launch the REPL to when you quit it.

Type in the following to define a `val` that will be used throughout the current REPL session:

```
val groceries = listOf("apples", "ground beef", "toilet paper")
```

You'll note that, since you've stored the output into a `val`, it doesn't immediately print out when you press Enter. If you don't assign to a `val` or `var`, the result of the print portion of the REPL will print directly to the console.

As an example, enter the following:

```
groceries.joinToString("\n")
```

Immediately, a newline-separated list of your groceries strings will print out:

```
apples  
ground beef  
toilet paper
```

You can perform any operation in the REPL that you can do with the standard library, including using lambdas and the `it` parameter. Enter the following to count how many characters are in each word of your groceries list:

```
groceries.map { it.count() }
```

Immediately, you'll see print out:

```
[6, 11, 12]
```

You can even define a class in the REPL. Enter the following to create a super-basic data class to hold the name and cost of a grocery item:

```
data class Grocery(val name: String, val cost: Float)
```

Press Enter. At the next prompt, create a list of groceries with their costs:

```
val moreGroceries = listOf(Grocery("apples", 0.50f),  
Grocery("ground beef", 5.25f), Grocery("toilet paper", 2.23f))
```

In addition to using the default `it` lambda parameter, you can also use Kotlin's ability to have named parameters in lambdas directly in the REPL.

Store the total cost into a variable by adding the following:

```
val cost = moreGroceries.fold(0.0f) { running, next ->  
    running + next.cost  
}
```

Finally, you can use Kotlin's string interpolation syntax the same way as you can in a normal class. Add the following to print out the total cost of your groceries:

```
println("Your groceries cost $cost")
```

When you press Enter, it will print out:

```
Your groceries cost 7.98
```

Now, exit the current REPL by typing:

```
:quit
```

When you press Enter, you'll see Terminal go back to its normal state rather than using `>>>` prompt in front of everything you're typing in. You can also press **Ctrl-D** on most systems to exit the REPL.

To see that nothing persists between sessions, launch the REPL again by typing:

```
kotlinc-jvm
```

When the REPL finishes launching and you see the `>>>` prompt again, try to print out the `groceries` constant from the previous session by entering:

```
println("$groceries")
```

When you press Enter, you'll see the following:

```
>>> println("Your groceries cost $cost")
Your groceries cost 7.98
>>> :quit
ellen@MacBook-Pro Book % kotlinc-jvm
Welcome to Kotlin version 1.4.21 (JRE 1.8.0_91-b14)
Type :help for help, :quit for quit
>>> println("$groceries")
error: unresolved reference: groceries
println("$groceries")
      ^
>>> █
```

error: unresolved reference: groceries

The error means that the REPL doesn't know what `groceries` is — which it shouldn't, since that was in the previous run of the REPL, before you typed in `:quit`. As long as a single REPL is running, it can hold information in memory and reference that information. But as soon as you quit that REPL, all the information it was holding disappears along with it.

Now, you can type `:quit` again to get out of the second REPL you started. Being able to try stuff out in the REPL is really helpful if you just want to investigate something quickly from the standard library. But what if you want to do something more complex — or make it reusable? This is where using files for your Kotlin scripts comes in.

Creating script files

Kotlin script files are a unique type of Kotlin file. They compile as if the entire file is the `main()` function for a Kotlin program.

This difference is emphasized by the fact that they have a different file extension. Normal Kotlin files end in a `.kt` extension and require a `main()` function to begin a running program. Kotlin script files, on the other hand, end in `.kts`. You'll read a bit more about when to use which one a bit later in the chapter.

You can run either type of file via either IntelliJ IDEA or the command line. In fact, compiling and running from the command line is actually what IntelliJ IDEA has been doing under the hood the entire time you've been using it. The user interface simply boils this down into the happy green Play button since that's much easier to understand.

Running via the command line might seem more complicated, but, as you'll see with `.kts` files, it's actually easy. So let's get started!

Running a script from the command line

First, go in your computer's file browser to the **starter** directory for this chapter. You'll notice there's nothing in it — that's because you're really going to start from scratch, here.

Open a new **Terminal** instance, and `cd` into the **starter** directory. Once there, run the following command to create a new, empty Kotlin script file:

```
touch script.kts
```

Leave your **Terminal** open, as you're going to come back to it once or twice. Open up the **script.kts** file that was just created in any text editor (e.g., **Sublime Text**, **Atom**, **VS Code**, **vim** or **emacs**) — you'll get to editing it with IntelliJ IDEA, shortly. Update the file to add the following line:

```
println("Hello, scripting!")
```

Note: If you're on macOS and using TextEdit, watch out that the system doesn't try to use "smart quotes" with your script because that can cause compilation issues. Make sure you're in text mode and not rich-text mode.

Save the **script.kts** file. Back in your **Terminal** window, enter the following command to run the script you just created:

```
kotlinc -script script.kts
```

Under the hood, the compiler will take your **.kts** file, compile it as if its contents were in a `main()` method that received the `args: List<String>` parameter, and then print out:

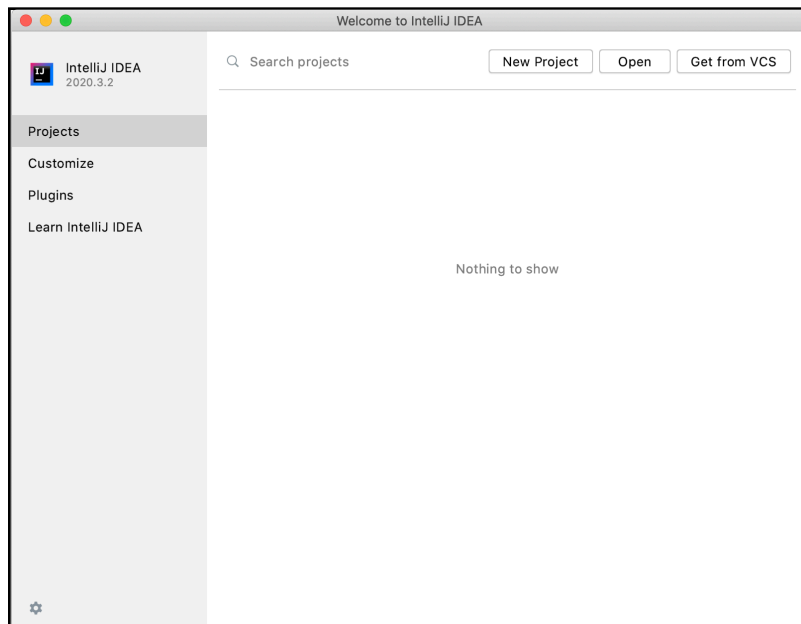
```
Hello, scripting!
```

Nice! But at this point, you're essentially back where you were with Python and Ruby. Easy to read and run, but you don't find out if anything is wrong until you fully compile the script.

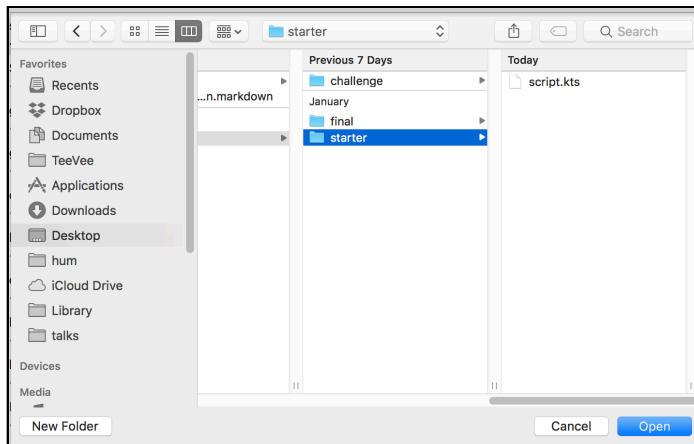
If you really want to combine the power of scripting and the safety of a compiled language, you'll want to edit the **.kts** file in the same IDE you've been using all along — IntelliJ IDEA.

Running a script with IntelliJ IDEA

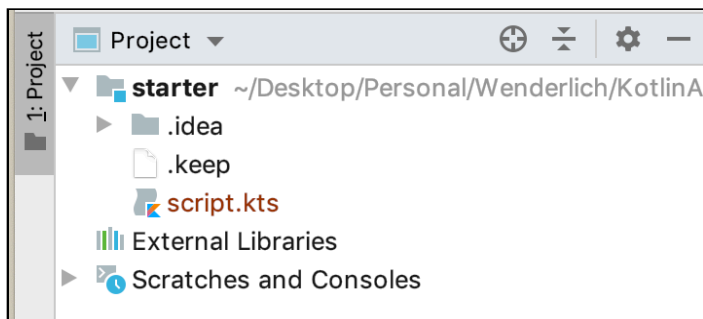
Quit your generic text editor and open up IntelliJ IDEA. If you close your other projects, you should land on this screen:



Select the **Open** option. You'll need to select the **starter** folder rather than the script itself:



Your project will open and, in the sidebar, you'll see that you can now see your `.kts` file:



Open up the file, and you'll see the same thing you had previously, but with nice pretty syntax highlighting:



Note: You may see an error on the `println` function that says “Unresolved reference: println.” Fear not, this will go away when you edit the project structure as you’ll read in a bit.

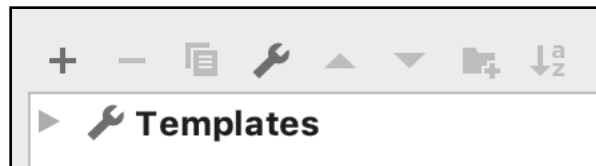
You'll also probably notice that, since there is no `main` method, there is no Play button next to your line. You know this runs from running it on the command line — but how do you run it in IntelliJ IDEA?

In the upper right-hand corner of IntelliJ IDEA, there's a button that says **Add configuration:**



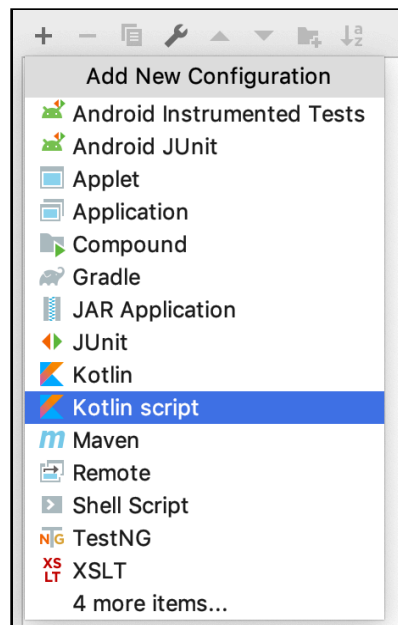
Click on it, and you'll see a screen where you can edit existing configurations or add new configurations that will run scripts for you.

In the top left-hand corner of this screen, there is a + button:

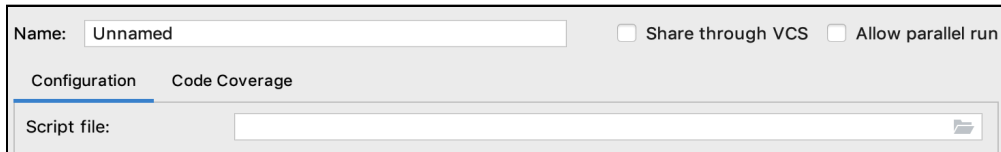


When you click on the +, a small window will pop up with some options for configurations. Select the **Kotlin script** option.

It will look like this:

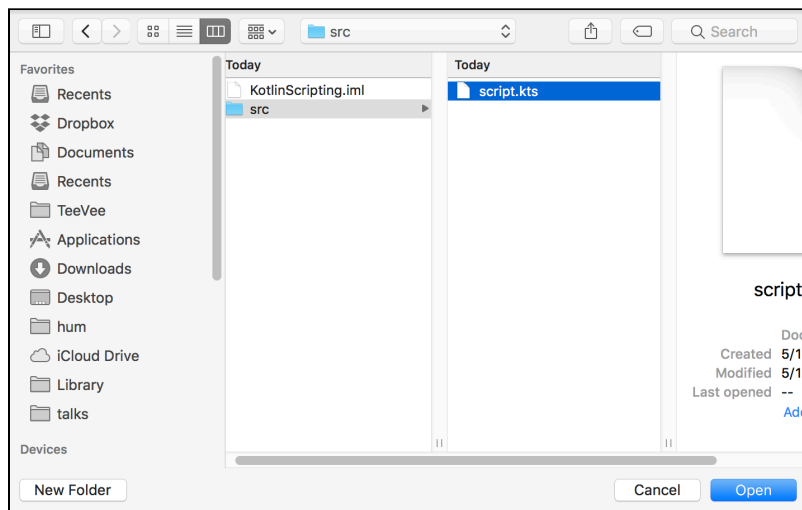


IntelliJ IDEA will create a configuration, and you'll be able to name it and select a script to run with it:



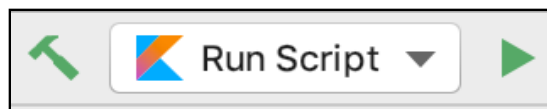
Name your configuration **Run Script**. Then, click the button on the far right of the **Script File:** text entry area. This will launch a file selector so you can choose what Kotlin script is run when this configuration is run.

Select your **script.kts** file and click **Open**:



Once you've selected the file, you'll get kicked back to the configuration editor, and the path to the selected file will be filled in. Click the **Apply** button on the configuration editor, then click the **OK** button.

This will close the configuration editor, and you'll see that the configuration you just created is now displayed next to the Run/Play button in the top-right of IntelliJ IDEA:

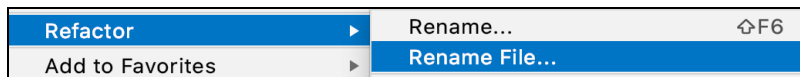


Click the Run button, and you'll see in the console:

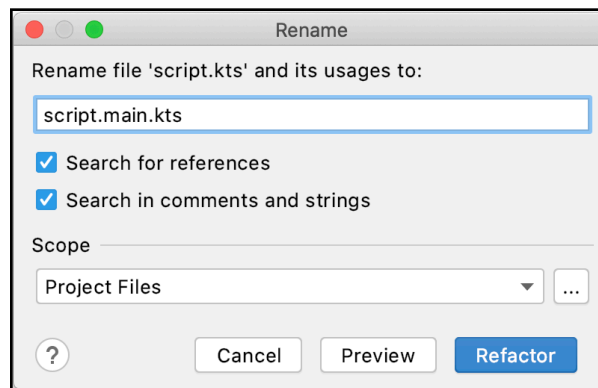
```
Hello, scripting!
```

One thing you may notice if you add to your script is that auto-completion is either very slow or not working at all – which defeats some of the point of using IntelliJ to edit the script. The good news is, a naming trick and some quick resets can fix this.

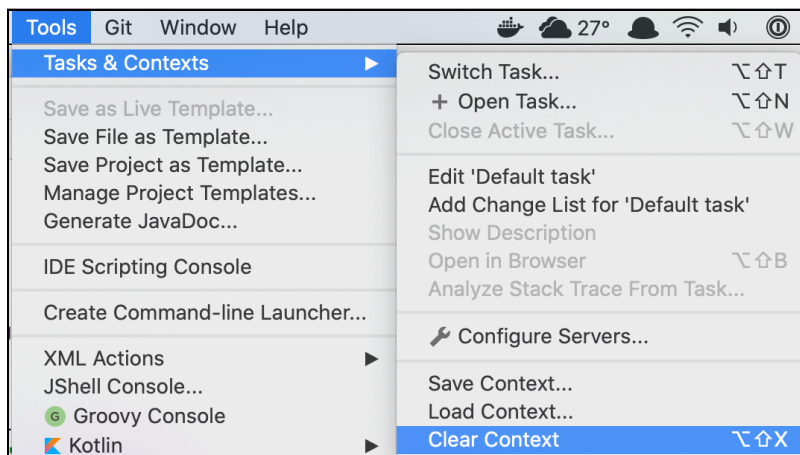
Select `script.kts`, right-click on it to bring up the context menu, and select **Refactor > Rename File**:



Update the name of the file to `script.main.kts` to help IntelliJ understand that this is a script meant to be run as the main of a program:



Click the **Refactor** button to finish renaming the file. Next, you'll need to manually clear the context of the script by going to **Tools > Tasks and Contexts > Clear Context**.



This will automatically close any open file. When you reopen `script.main.kts`, you should see a banner at the top that lets you know a new script context is available:

There is a new script context available.

[Apply context](#) [Enable auto-reload](#)

Note: If this banner doesn't show up immediately, you may need to clear the context again – it can be a little tempermental.

Click **Apply Context**, and you'll now get syntax highlighting and autocomplete in your script without issue.

Woo hoo! Now it's time to figure out how to give your script the information it needs to be flexible: User input!

Handling arguments

An **argument**, when it comes to running a Kotlin script, is a string that you enter into the command line. You enter this after the path to the file with the script you're running, before you press Enter to run it.

You can separate multiple arguments with spaces, and they'll automatically be turned into a `List`. If you want to pass in a string with spaces as a single argument, you can put it in quotation marks.

As you'll recall, a `.kts` file essentially treats the entire contents of the file as the contents of a `fun main(args: List<String>)` method. If you'd been wondering what `args` is short for, now you know: arguments!

Even though you don't see the `args` declaration directly in your script, you can still access the arguments from the script.

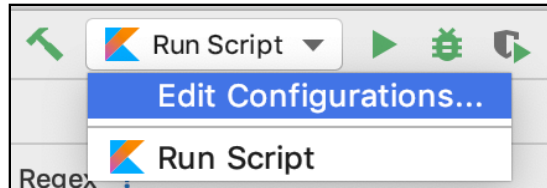
Add the following lines to **script.main.kts** to print out a list of the arguments if any were received; this will also let you know if there weren't any arguments received:

```
if (args.isEmpty()) {  
    println("[no args]")  
} else {  
    println("Args:\n ${args.joinToString("\n ")})"  
}
```

Save the file, then click the Run button again. You'll see the script print out:

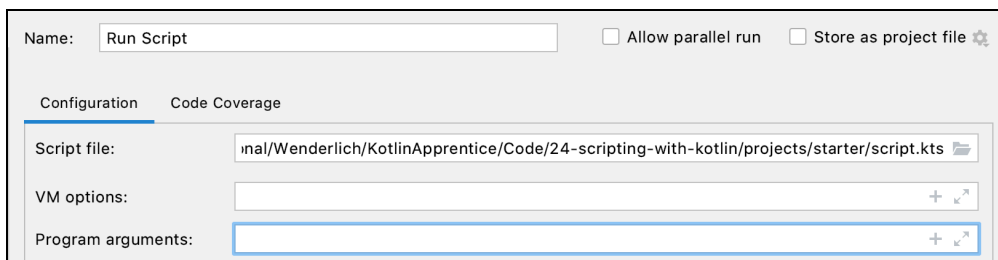
```
[no args]
```

So, how do you add arguments when running in IntelliJ IDEA? By using the configuration you created earlier. Click the drop-down where the run script is displayed, and select **Edit Configurations...**:



You'll see the configuration editor, and your current configuration — the one you created earlier — is selected by default.

You'll notice that one of the inputs is named **Program arguments**:



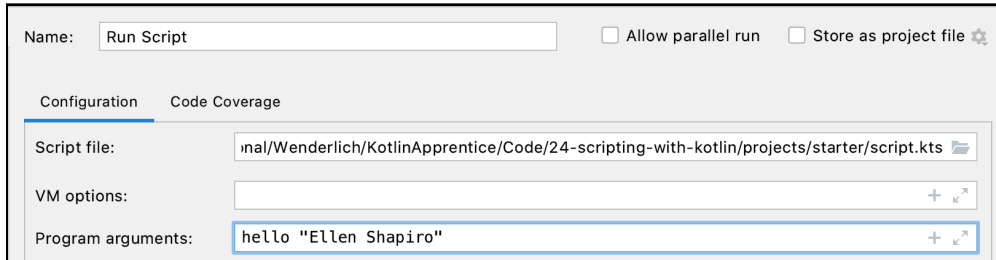
Into this input line, type:

```
hello
```

Then click the **OK** button to save your changes and dismiss the configuration editor. Run the configuration again, and you'll see the following included in the console output:

```
Args:  
hello
```


Excellent! Now, go back and select **Edit Configurations...** again. Update the program arguments to include your first and last name as a single argument, by putting both in quotation marks:



Run the program again, and you'll see:

```
Args:  
hello  
Ellen Shapiro
```

...although it'll be with your name instead of "Ellen Shapiro"!

You've probably noticed that making all these changes to the configuration is a real hassle. One of the ways you can improve the iteration time for changing parameters on your script is to continue to run it using the command line.

Remember that Terminal window you had open earlier, where you'd `cd`'d into the starter folder? Find that window again — or open a new Terminal window and `cd` into the starter folder again if you can't find the one you used before.

Previously, you ran `kotlinc -script script.kts` to run your script, but remember you changed the filename. Enter the same command using the new filename:

```
kotlinc -script script.main.kts
```

Press Enter. Here, it's much clearer that there are no arguments passed in, since there's nothing after the `script.main.kts` path. Indeed, the following will print:

```
[no args]
```

Press the Up Arrow key to bring up the last command, then type in some additional parameters, as shown here:

```
kotlinc -script script.main.kts Kotlin scripting is awesome
```

Press Enter to run it again, and you'll see:

```
Args:
Kotlin
scripting
is
awesome
```

Now, press the Up Arrow again. Use the Left and Right Arrow keys to go back and forth at the command line, adding quotes around the four arguments to turn them into one single argument:

```
kotlinc -script script.main.kts "Kotlin scripting is awesome"
```

Now, your output will only have a single argument:

```
Args:
Kotlin scripting is awesome
```

As you can see, you get a much clearer and faster iteration than having to muck around with a configuration file. For the rest of these examples, you should keep IntelliJ IDEA open as a text editor to work on the script, so that you get the benefits of compile-time type checking. But you should run the program using a Terminal shell, after saving any changes using **File ▶ Save All** or pressing **Command-S** on Mac or **Ctrl-S** on PC.

Now, it's time to try to do something a little more useful: get and print out information about the filesystem.

Getting information from the system

Getting information about the filesystem is really helpful because you can use it in many different ways: moving files around, copying files, and figuring out how large files are or where they're located.

You're going to do something relatively simple here: print out the names of the files in a passed-in folder. It's a great example of how you can work with existing Kotlin and Java APIs for something that runs at the command line.

Even though with your script you're effectively in the `main()` function of a program, you can still add other functions and use things from the standard library really easily.

Here, since you're working with the filesystem, you're going to want to take advantage of some functionality which is built into the JDK: Its handling of files and folders through the **File** class.

File is actually a little bit misleading as a name for this class, because a File object could be either a file or a folder. In fact, it's quite easy using File to get the current working directory, which is a folder.

In **script.main.kts**, add a function below your argument parser:

```
fun currentFolder(): File {  
    return File("").absoluteFile  
}
```

You'll notice that, when you add the File return type, IntelliJ IDEA might automatically import the Java class at the top of your **script.main.kts** file:

```
import java.io.File
```

If it does not, you should be able to press **Option-Return** on Mac or **Alt-Enter** on PC to pull in the import statement. Even though you're already in a `main()` function, you can still import the libraries you need to make everything work. Neat!

Since the method you created passes in an empty string to the File object, by default, it will return the current working directory from which this script is called. Add a couple of lines to print the name of the current folder:

```
val current = currentFolder()  
println("Current folder: $current")
```

Save the script in IntelliJ IDEA and then run the script again by pressing the Up Arrow key in Terminal and pressing Enter. At the bottom of the printout, you'll see (with `[fullpath]` replaced by the full path to wherever you've put the *Kotlin Apprentice* code):

```
Current folder: [fullpath]/KotlinApprentice/24-scripting-with-  
kotlin/projects/starter
```

Excellent — your script now knows where you are. Now, it's time to find out a bit more about what's around you.

You've seen that you can add functions within a `.kts` script even though it's already inside a `main()` function. You can also use extension functions to add functionality to existing Kotlin or Java classes.

Add an extension function to `File` to get a `List<File>` of the contents of the current `File` object, which, as a reminder, is actually a folder:

```
fun File.contents(): List<File> {  
    return this.listFiles().toList()  
}
```

Update your line printing out the folder to the following, printing out the folder's contents instead, then save the script:

```
val current = currentFolder()  
println("Current folder contents:\n $  
{current.contents().joinToString("\n ")}")
```

Go back to Terminal and press the Up key to bring up the previous command, then press Enter. You'll see something like this:

```
Current folder contents:  
[fullpath]/KotlinApprentice/scripting-with-kotlin/projects/  
starter/.DS_Store  
[fullpath]/KotlinApprentice/scripting-with-kotlin/projects/  
starter/script.main.kts  
[fullpath]/KotlinApprentice/scripting-with-kotlin/projects/  
starter/.idea
```

Note: If you're not on a system using macOS, you won't see the `.DS_Store` file — that's a type of file that's specific to the Mac filesystem.

If you've got your folder for this book buried deep in your filesystem, you'll probably realize that this is not giving you exceptionally useful information since there's so much other noise being printed.

Instead, below your extension function to get the `contents()` of a folder, add another extension function to get just the names of the files within the folder:

```
fun File.fileNames(): List<String> {  
    return this.contents().map { it.name }  
}
```

Next, update the line printing out the contents of the folder to use this new method:

```
println("Current folder contents:\n $  
{current.fileNames().joinToString("\n ")})")
```

Save the script, go back to Terminal and press the Up key again to bring up the previous command, then press Enter. You'll now see something far shorter than you were seeing previously:

```
Current folder contents:  
  .DS_Store  
  script.main.kts  
  .idea
```

Aha! Now, it's just the file names. But there aren't just file names in there — there are also folder names. Again, a `File` object could be a file or a folder. So how do you tell the difference?

`File` has two convenience properties to help with this: `isDirectory` and `isFile`. Using those, create two new extension methods to list out the folders and the files of a given `File` object:

```
fun File.folders(): List<File> {  
    return this.contents().filter { it.isDirectory }  
}  
  
fun File.files(): List<File> {  
    return this.contents().filter { it.isFile }  
}
```

Update your `fileNames()` extension method to only return the names of files:

```
fun File.fileNames(): List<String> {  
    return this.files().map { it.name }  
}
```

Next, add another extension method that only returns the names of folders:

```
fun File.folderNames(): List<String> {  
    return this.folders().map { it.name }  
}
```

Now, it's time to pull everything into a convenience method so that you can access it more easily.

Add the following method:

```
fun File.printFolderInfo() {  
    // 1  
    println("Contents of `${this.name}`:")  
  
    // 2  
    if (this.folders().isEmpty()) {  
        println("- Folders:\n    `${this.folderNames().joinToString("\n    ")}`")  
    }  
  
    // 3  
    if (this.files().isEmpty()) {  
        println("- Files:\n    `${this.fileNames().joinToString("\n    ")}`")  
    }  
  
    // 4  
    println("Parent: `${this.parentFile.name}`")  
}
```

What's going on here?

1. You print the name of the current folder, so you know what you're printing information about.
2. You print a list of the names of the folders within the current folder, if there are any.
3. You print a list of the names of the files within the current folder, if there are any.
4. You print information about the parent File object — which, again, is actually a folder.

Now, remove the existing line in your code printing information about current and replace it with a line that calls this new convenience method:

```
current.printFolderInfo()
```

Save the script, go back to Terminal, press the Up key once more to bring up the last command, and then press Enter. You should see something like:

```
Contents of `starter`:  
- Folders:  
  .idea  
- Files:  
  .DS_Store  
  script.main.kts
```

Parent: projects

Now that you're successfully printing the contents of the current folder, it's time to start using the power of command line arguments so that you can print the information of any arbitrary folder in your system!

Adding arguments

Often, arguments are passed in with the format `name=Value`. This makes it easy to detect which argument is for what purpose, no matter what the order of arguments is.

At the bottom of **script.main.kts**, create a function to check for an argument prefix, then return the value for that argument if something was actually passed in for the argument:

```
fun valueFromArgsForPrefix(prefix: String): String? {
    val arg = args.firstOrNull { it.startsWith(prefix) }

    if (arg == null) return null

    val pieces = arg.split("=")
    return if (pieces.size == 2) {
        pieces[1]
    } else {
        null
    }
}
```

Next, below the function `valueFromArgsForPrefix()`, add some lines to look for a particular prefix for an argument:

```
val folderPrefix = "folder="
val folderValue = valueFromArgsForPrefix(folderPrefix)
```

Then, add code to print info about either the passed-in folder or the current working directory if no folder was passed in:

```
if (folderValue != null) {
    val folder = File(folderValue).absoluteFile
    folder.printFolderInfo()
} else {
    println("No path provided, printing working directory info")
    currentFolder().printFolderInfo()
}
```

Next, save the script then go back to **Terminal** and type in:

```
pwd
```

Press Enter. This will print out something similar to:

```
/Users/ellen/Desktop/Wenderlich/KotlinApprentice/scripting-with-  
kotlin/projects/starter
```

It will, however, have a different prefix based on the location of the code for this book depending on where you put it on your hard drive. Copy this value by selecting it and pressing **Command-C** on Mac or **Ctrl-Shift-C** on PC.

Next, press the Up button on your keyboard twice — once to get `pwd` and then another time to get your parameters back. You should see:

```
kotlinc -script script.main.kts "Kotlin scripting is awesome"
```

Press Enter to see what happens now if you don't pass a folder in, and you'll see the same folder information you saw before:

```
No path provided, printing working directory info  
Contents of `starter`:  
- Folders:  
  .idea  
- Files:  
  .DS_Store  
  script.main.kts  
Parent: projects
```

Now, it's time to actually pass in a folder. Press Up one more time to bring up the command and parameters. This time, add the folder parameter by typing `folder=`, then pasting in the path you printed out earlier of the current working directory.

You should see something like this, though with your own path to the *Kotlin Apprentice* folder:

```
kotlinc -script script.main.kts "Kotlin scripting is awesome"  
folder=/Users/ellen/Desktop/Wenderlich/KotlinApprentice/  
scripting-with-kotlin/projects/starter
```


Delete the `/starter` at the end of this line so that the `folder` parameter ends with `/projects` — this will give you the contents of the `projects` folder. Press Enter again, and you'll see:

```
Contents of `projects`:  
- Folders:  
  starter  
  final  
  challenge  
- Files:  
  .DS_Store
```

You can continue altering the path for the folder to be whatever you want it to be — and the contents of the folder will always print out. You'll run into an issue if you try to pass in the root folder of your filesystem using `folder=/`; see if you can find the cause of the error in the script and fix it.

Congratulations! You've now learned how to write a Kotlin script, how to run it from both IntelliJ IDEA and the command line, and how to make your script do different things based on the arguments passed in when the script is run.

Challenges

1. In the script you created to list the contents of a directory, add a way to decide from a passed-in parameter whether hidden files (i.e., files that start with a `.` and so are not normally rendered visible in your filesystem browser) should be included in the list of things printed out or not. You should default to not showing hidden files in the list.
2. Create a Kotlin script to take a string and change its letters by using a **ROT-n** encoder: offset each letter by `n` places in the 26-letter English alphabet, then print out the scrambled string.
3. In your ROT-n script, add handling for an argument that tells you how many letters your script should rotate by.
4. Figure out a way to call your ROT-n script repeatedly in order to get back the same string you put in. No, you may not use ROT-26, smarty-pants.

Key points

- **Scripting** is writing small programs in a text editor that can be run from the command line and be used to do various types of processing on your computer.
- As a scripting language, Kotlin gives you the **static-typing** lacking in other scripting languages like Python and Ruby.
- Kotlin comes with a **Read-Evaluate-Print Loop** or REPL that can be used to investigate Kotlin code in an interactive manner.
- Kotlin scripts end with the extension `.kts`, as opposed to normal Kotlin code that ends with `.kt`.
- You can use IntelliJ IDEA as a script editor, and then either run your scripts within the IDE or from a command line shell on your OS.
- Kotlin scripts run inside a hidden `main()` function and can access `args` passed into the script at the command line. Scripts can also import Kotlin and Java libraries to access their features.
- You can use Kotlin scripts to read and write to the files and folders on your filesystem, and much more!

Where to go from here?

The tool `kscript` (<https://github.com/holgerbrandl/kscript>) provides a convenient wrapper around Kotlin scripting. It allows pre-compilation of scripts, which results in much faster iteration, and it also allows you to use a simpler syntax for accessing Kotlin at runtime. The creator gave a talk at KotlinConf 2017 which is worth watching for a great outline of some of the problems he was trying to solve. You can watch it at <https://www.youtube.com/watch?v=cOJPKhIRa8c>.

In the next chapter, you'll see an alternative to Kotlin scripting: building native command-line programs with Kotlin. You'll also begin to prepare yourself to work with Kotlin on multiple platforms!